

um-game-playing-strength-of-mcts-variants

11位解法

Rank	11
Team	Kansai-kaggler
Author	RYUSHI
Topic ID	549708
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549708>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Team: kansai-kaggler ([@tmhrkt](#)、[@yystsuji](#)、[@ryushisa](#))

素晴らしいコンペを開催したKaggle Staffとホストの皆さんに心より感謝します。私たちの解法を共有します。

概要

3人がそれぞれ単体モデルを作り、それらをstackingする戦略を採りました。

CV戦略

ゲーム実施総数で層化したStratifiedGroupKFoldを使いました。Discussionで指摘されたように、実施ゲーム数によって分布が変わることを懸念したためです。

GameRulesetName と EnglishRules

ホストがテストには未知の GameRulesetName が含まれると明言したため、そのTF-IDFは危険です。

EnglishRules もLudus Coriovalliのvariantなど記述がノイズでした。そのため両方を特徴量に使いませんでした。

Ryushi part

[UM] [Gradient Boosting Models Train&Infer](#) を基にしました。LightGBM、XGBoost、CatBoostを容易に実験できる非常に有用なNotebookです。最終的にはCatBoostだけが最良でした。

特徴量

最良単体モデルは約180特徴量だけを使いました。

削除したもの：一意値1の特徴量、欠損95%超の特徴量、一部を除くfrequency系と `_components` 列。分布を見て情報量が妥当に見えるものは残しました。

- 残したfrequency: DrawFrequency, StepDecisionFrequency, FromToDecisionEmptyFrequency, SetNextPlayerFrequency, RemoveEffectFrequency, MoveAgainFrequency, StepDecisionToEmptyFrequency
- 残したcomponents: NumStartComponentsBoard, NumStartComponentsHand, NumStartComponents, NumStartComponentsBoardPerPlayer, MarkerComponent

多重共線性も考慮しました。例えば相互相関90%超の ScoreDifferenceVariance, ScoreDifferenceMedian, ScoreDifferenceMaximum, ScoreDifferenceChangeAverage, ScoreDifferenceMaxDecrease, ScoreDifferenceAverage, ScoreDifferenceMaxIncrease, ScoreDifferenceChangeLineBestFit から ScoreDifferenceVariance だけを残しました。同様の処理

が必要な列は複数ありました。この特徴量選択とparameter tuningでPublic LBはbaseline 0.464から単体0.422へ改善しました。

追加したもの：LudRules の長さ、start・play・end section由来の特徴量、LudRules 内の Draw・Win・Loss の出現回数。timeout以外にもゲーム自体がdrawを定めるpatternがあるという仮説です。私のpartは特徴量が少なく、他memberは多かったため、最終的な多様性に寄与したと考えます。

exp	CV	Public	Private
057	0.4092	0.422	0.432
058	0.4087	0.422	0.432
059	0.4070	0.422	0.432
069	0.4082	0.424	0.432
071	0.4079	0.424	0.432

効果がなかったのはNNと、CatBoost MultiRegressionで utility_agent1 と draw_ratio を同時予測する方法です。多数の実験をしましたが、結局中盤の実験が機能し、複雑な特徴量・モデルは精度を下げました。

ktm part

- Augmentation: agent1/agent2を交換し、1-AdvantageP1、-utility_agent1 とswapped flagを追加。推論時はTTA。
- Multi target: CatBoost MultiRMSEで utility_agent1 と draw_ratio を予測。
- Data generation: Ludii Portalの公開ruleを使い、Colab CPUとvast.ai instanceでtrainにない30k〜40k行を生成。CVは改善したがLBは不変。
- Target encoding。

exp	CV	LB	Private	target enc	Multi Target	extra data
084	0.4015	0.429	0.434	✓	✓	
088	0.4006	0.427	0.433	✓	✓	✓
094	0.4034	0.430	0.436	✓		✓
101	0.4016	0.429	0.435		✓	✓

2g part

MCTS Starterを基に、次の適用有無を変えた複数モデルを用意しました。

- agent1/agent2を交換したデータを追加。
- binary classificationでdrawを予測し、drawらしい予測の utility_agent1 を0にする。
- LudRules を start・play・end に分割し、各々へTF-IDF。

- sectionごとの `depth_nest` (`()`・`{}` によるnest深さ)、`num_condition` (if 数)、`chr_len`、`unique_start_pieces` (place 数) などを作成。

exp	CV	Public	Private
017-1	0.4092	unknown	unknown
018-1	0.4073	0.422	0.431
024-1	0.4070	0.422	0.431
025-1	0.4094	0.422	0.430

Stacking (Ryushi & ktm)

ここで大きく改善しました。fold間でCVが大きく変動し、model seedだけを変えても予測が変わる問題があったため、単純ensembleよりstackingが有効と考えました。foldごとにscoreが大きく違うため、各foldの重み（特徴量）をOptunaで最適化しました。

最良stackingはCV 0.3912 / Public 0.419 / Private 0.424で、Ryushiの5実験、2gの4実験、ktmの7種類（draw modelを含む）を候補に、foldごとに異なるsubsetを選びました。fold別CVは0.3977、0.3677、0.38142、0.40944、0.39866でした。

最終提出の選択

ゲーム実施総数でfold分割しましたがCVとLBの良い相関を得られず、次の2つを選びました。

- **CV+LB Best:** stacking群の中でCVとLBの双方が良いもの。
- **CV Best:** Nelder–Meadで重みを最適化した提出のうち最良CV。過学習を懸念しLB Bestは選ばず。

PrivateはPublicと相関し、選んだCV+LB BestがPrivateでも最良でした。

Tips: Nelder–Mead

各submissionの重みをNelder–Meadで最適化しました。重み合計を1に制約しないとCV/LBが改善し、その合計は約1.14でした。他解法が述べたように、予測へ定数を掛けることが重要だった可能性があります。